

5장. 데이터 사전 처리

컴퓨터공학과 전승현 교수



Contents



01

누락 데이터 처리

02

중복 데이터 처리

03

데이터 표준화

04

범주형 (카테고리) 데이터 처리

Contents

05

정규화

06

시계열 데이터



누락 데이터 처리

※ 출처: <https://www.kaggle.com/datasets/yasserh/titanic-dataset?resource=download>

- NaN Not a Number

- 누락 데이터 확인

- 타이타닉 데이터에서 누락 데이터 확인

✓
2초

```
[1] import seaborn as sns  
  
df = sns.load_dataset('titanic')  
df.head()
```



	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True



누락 데이터 처리

- NaN^{Not a Number}

- 누락 데이터 확인

- NaN 카운트를 확인하려면?



0초

[2]



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    object
3   age         714 non-null    float64
4   sibsp       891 non-null    int64
5   parch       891 non-null    int64
6   fare        891 non-null    float64
7   embarked    889 non-null    object
8   class       891 non-null    category
9   who         891 non-null    object
10  adult_male  891 non-null    bool
11  deck        203 non-null    category
12  embark_town 889 non-null    object
13  alive       891 non-null    object
14  alone       891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

누락 데이터 처리

- NaN^{Not a Number}

- 누락 데이터 확인

- NaN 카운트를 확인하려면?



0초

```
[2] df.info()
```



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   survived              891 non-null    int64
1   pclass                891 non-null    int64
2   sex                   891 non-null    object
3   age                   714 non-null    float64
4   sibsp                 891 non-null    int64
5   parch                 891 non-null    int64
6   fare                  891 non-null    float64
7   embarked              889 non-null    object
8   class                 891 non-null    category
9   who                   891 non-null    object
10  adult_male            891 non-null    bool
11  deck                  203 non-null    category
12  embark_town           889 non-null    object
13  alive                  891 non-null    object
14  alone                 891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

688 개 Null 값 (= 891 - 203)

누락 데이터 처리

- NaN^{Not a Number}

- 누락 데이터 확인

- NaN 카운트를 확인하는 다른 방법

✓
0초

```
[3] nan_deck = df['deck'].value_counts(dropna=False)  
     print(nan_deck)
```

deck 자체 데이터를 group by함



```
deck  
NaN    688  
C       59  
B       47  
D       33  
E       32  
A       15  
F       13  
G        4  
Name: count, dtype: int64
```

누락 데이터 처리

- NaN (Not a Number)

- 누락 데이터 확인

- NaN 카운트를 확인하는 다른 방법

✓
0초 [7] `print(df.isnull().sum(axis = 0))`

⇒

survived	0
pclass	0
sex	0
age	177
sibsp	0
parch	0
fare	0
embarked	2
class	0
who	0
adult_male	0
deck	688
embark_town	2
alive	0
alone	0

dtype: int64

✓
0초 [8] `print(df.head().isnull().sum(axis = 0))`

⇒

survived	0
pclass	0
sex	0
age	0
sibsp	0
parch	0
fare	0
embarked	0
class	0
who	0
adult_male	0
deck	3
embark_town	0
alive	0
alone	0

dtype: int64

누락 데이터 처리

- NaN^{Not a Number}

- 누락 데이터 확인

- NaN을 확인하는 다른 방법

✓
0초

```
[4] print(df.head().isnull())
```



	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	#
0	False	False	False	False	False	False	False	False	False	
1	False	False	False	False	False	False	False	False	False	
2	False	False	False	False	False	False	False	False	False	
3	False	False	False	False	False	False	False	False	False	
4	False	False	False	False	False	False	False	False	False	

	who	adult_male	deck	embark_town	alive	alone
0	False	False	True	False	False	False
1	False	False	False	False	False	False
2	False	False	True	False	False	False
3	False	False	False	False	False	False
4	False	False	True	False	False	False

누락 데이터 처리

- NaN^{Not a Number}

- 누락 데이터 확인

- NaN을 확인하는 다른 방법

✓
0초

```
[5] print(df.head().notnull())
```



	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	#
0	True	True	True	True	True	True	True	True	True	True	
1	True	True	True	True	True	True	True	True	True	True	
2	True	True	True	True	True	True	True	True	True	True	
3	True	True	True	True	True	True	True	True	True	True	
4	True	True	True	True	True	True	True	True	True	True	

	adult_male	deck	embark_town	alive	alone
0	True	False	True	True	True
1	True	True	True	True	True
2	True	False	True	True	True
3	True	True	True	True	True
4	True	False	True	True	True

누락 데이터 처리

- 누락 데이터 제거

- 열 또는 행 삭제 방법

- 열: 특징 변수 제거
 - 행: 관측값 (레코드) 제거

✓
0초

```
[3] missing_df = df.isnull()
    for col in missing_df.columns:
        missing_count = missing_df[col].value_counts().get(1, 0)

        try:
            print(col, ': ', missing_count[True])
        except:
            print(col, ': ', 0)
```

```
survived : 0
pclass : 0
sex : 0
age : 177
sibsp : 0
parch : 0
fare : 0
embarked : 2
class : 0
who : 0
adult_male : 0
deck : 688
embark_town : 2
alive : 0
alone : 0
```

누락 데이터 처리

- 누락 데이터 제거

- 열 또는 행 삭제 방법

- 열: 특징 변수 제거
 - 행: 관측값 (레코드) 제거

✓
0초

```
[3] missing_df = df.isnull()
    for col in missing_df.columns:
        missing_count = missing_df[col].value_counts()

        try:
            print(col, ': ', missing_count[True])
        except:
            print(col, ': ', 0)
```

```
survived : 0
pclass : 0
sex : 0
age : 177
sibsp : 0
parch : 0
fare : 0
embarked : 2
class : 0
who : 0
adult_male : 0
deck : 688
embark_town : 2
alive : 0
alone : 0
```

누락 데이터 처리

- 누락 데이터 제거

- 열 삭제

- NaN 값이 500개 이상 되는 모든 열 (axis = 1) 삭제

✓
0초

```
[4] df_thresh = df.dropna(axis = 1, thresh = 500)  
     print(df_thresh.columns)
```



```
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',  
       'embarked', 'class', 'who', 'adult_male', 'embark_town', 'alive',  
       'alone'],  
      dtype='object')
```

누락 데이터 처리

- 누락 데이터 제거

- 행 삭제

- 특정 열을 한정하여 해당 행에 NaN 값이 있는 경우 모든 행 (axis = 0) 삭제

✓
0초

```
[6] df_age = df.dropna(subset = ['age'], how = 'any', axis = 0)  
     print(len(df_age))
```

⇒ 714

891 - 177

✓
0초

```
[3] missing_df = df.isnull()  
     for col in missing_df.columns:  
         missing_count = missing_df[col].value_counts()  
  
         try:  
             print(col, ': ', missing_count[True])  
         except:  
             print(col, ': ', 0)
```

⇒

```
survived : 0  
pclass : 0  
sex : 0  
age : 177  
sibsp : 0  
parch : 0  
fare : 0  
embarked : 2  
class : 0  
who : 0  
adult_male : 0  
deck : 688  
embark_town : 2  
alive : 0  
alone : 0
```

누락 데이터 처리

- 누락 데이터 치환

- age의 NaN값 평균값으로 치환하기

- fillna() 메소드

✓
0초

```
[20] import seaborn as sns

df = sns.load_dataset('titanic')

print(df['age'].head(10))
print('\n')

mean_age = df['age'].mean(axis = 0)
df['age'].fillna(mean_age, inplace = True)

print(df['age'].head(10))
```



0	22.0
1	38.0
2	26.0
3	35.0
4	35.0
5	NaN
6	54.0
7	2.0
8	27.0
9	14.0

Name: age, dtype: float64

0	22.000000
1	38.000000
2	26.000000
3	35.000000
4	35.000000
5	29.699118
6	54.000000
7	2.000000
8	27.000000
9	14.000000

Name: age, dtype: float64

누락 데이터 처리

- 누락 데이터 치환

- embark_town (승선 도시)의 NaN값을 다른 값으로 치환하기
 - 가장 많이 승선한 도시 이름을 찾아 치환함

✓
0초

```
[22] print(df['embark_town'][825:830])  
      print('\n')  
  
      most_freq = df['embark_town'].value_counts(dropna = True).idxmax()  
      print(most_freq)  
      print('\n')  
  
      df['embark_town'].fillna(most_freq, inplace = True)  
      print(df['embark_town'][825:830])
```

최대값을 가지는 칼럼명

```
825    Queenstown  
826    Southampton  
827     Cherbourg  
828    Queenstown  
829         NaN  
Name: embark_town, dtype: object
```

Southampton

```
825    Queenstown  
826    Southampton  
827     Cherbourg  
828    Queenstown  
829    Southampton  
Name: embark_town, dtype: object
```


누락 데이터 처리

- 누락 데이터 치환

- 만약 NaN값이 다른 형태로 보이는 경우

- `df.replace('?', np.nan, inplace = True)`

`import numpy as np`

- 실제로 RDBMS 상에서 Null값 속성으로 지정된 칼럼에 들어오는 데이터는 ' ' (space) 데이터도 많음 //

Null값인데 무의미한 값이 존재하여 데이터 분석에 방해가 됨

누락 데이터 처리

- 누락 데이터 치환

- embark_town의 바로 직전 값으로 치환하기

- method = 'ffill' 옵션

[15]
✓ 0초

```
import seaborn as sns

df = sns.load_dataset('titanic')

print(df['age'].head(10))
print('\n')

mean_age = df['age'].mean(axis = 0)
df['age'].fillna(mean_age, inplace = True)

print(df['age'].head(10))
```

다시 실행해야 함

[17]
✓ 0초

```
print(df['embark_town'][825:830])
print('\n')

df['embark_town'].fillna(method = 'ffill', inplace = True)
print(df['embark_town'][825:830])
```

▼

```
825    Queenstown
826    Southampton
827     Cherbourg
828    Queenstown
829         NaN
Name: embark_town, dtype: object
```

```
825    Queenstown
826    Southampton
827     Cherbourg
828    Queenstown
829    Queenstown
Name: embark_town, dtype: object
```

중복 데이터 처리

- 중복 데이터 확인

- 행의 레코드 중복 여부 확인

- duplicated() 메소드

✓
0초

```
[24] import pandas as pd

df = pd.DataFrame({'c1': ['a', 'a', 'b', 'a', 'b'],
                  'c2': [1, 1, 1, 2, 2],
                  'c3': [1, 1, 2, 2, 2]})

print(df)
print('\n')

df_dup = df.duplicated()
print(df_dup)
print('\n')
```



	c1	c2	c3
0	a	1	1
1	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

0	False
1	True
2	False
3	False
4	False

dtype: bool

중복 데이터 처리

- 중복 데이터 확인

- 열의 레코드 중복 여부 확인

- uplicated() 메소드를 바로 못씀

```
✓  
0초 [25] col_dup = df['c2'].duplicated()  
      print(col_dup)
```

```
⇒ 0    False  
   1     True  
   2     True  
   3    False  
   4     True  
   Name: c2, dtype: bool
```

⇒	c1	c2	c3
0	a	1	1
1	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

중복 데이터 처리

- 중복 데이터 제거

- 데이터프레임에서 중복된 행 제거

- 중복된 1 행 제거

✓
0초

```
[26] print(df)
      print('\n')

      df2 = df.drop_duplicates()
      print(df2)
```



	c1	c2	c3
0	a	1	1
1	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

	c1	c2	c3
0	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

중복 데이터 처리

- 중복 데이터 제거

- 데이터프레임에서 특정 열에 대한 중복된 행 제거

- 중복된 1, 4 행 제거

✓
0초

```
[29] print(df)
      print('\n\n')

      df3 = df.drop_duplicates(subset = ['c2', 'c3'])
      print(df3)
```



	c1	c2	c3
0	a	1	1
1	a	1	1
2	b	1	2
3	a	2	2
4	b	2	2

	c1	c2	c3
0	a	1	1
2	b	1	2
3	a	2	2

데이터 표준화

- 단위 환산

- 영미권 측량 기준과 한국은 다른 부분이 있음

- 영미권: 연비를 갤런당 마일mile per gallon로 표시
 - 한국: 연비를 리터당 킬로미터km/L로 표시

```
✓ [31] !pip install ucimlrepo
```

```
✓ [32] from ucimlrepo import fetch_ucirepo

# fetch dataset
auto_mpg = fetch_ucirepo(id=9) # auto mpg

# data (as pandas dataframes)
X = auto_mpg.data.features
y = auto_mpg.data.targets
```

```
✓ [34] mpg_to_kpl = 1.60934/3.78541 # 1마일 = 1.60934킬로, 1갤런 = 3.78541리터
      y['kpl'] = y * mpg_to_kpl
      print(y.head(3))
      print('\n')

      y['kpl'] = y['kpl'].round(2)
      print(y.head(3))
```

```
↔
      mpg      kpl
0  18.0  7.652571
1  15.0  6.377143
2  18.0  7.652571
```

```
      mpg      kpl
0  18.0  7.65
1  15.0  6.38
2  18.0  7.65
```

데이터 표준화

※ 출처: <https://www.kaggle.com/datasets/uciml/autompg-dataset>

• 자료형 변환

▪ 숫자가 문자열object로 저장된 경우가 있음

- auto-mpg.csv 를 colab으로 업로드하고 df로 read_csv() 하고
- 숫자 데이터에 문자열 데이터가 있는지 확인
- horsepower 속성은 object임

```
✓ [39] print(df['horsepower'].unique())
```

```
↔ ['130' '165' '150' '140' '198' '220' '215' '225' '190' '170' '160' '95'  
   '97' '85' '88' '46' '87' '90' '113' '200' '210' '193' '?' '100' '105'  
   '175' '153' '180' '110' '72' '86' '70' '76' '65' '69' '60' '80' '54'  
   '208' '155' '112' '92' '145' '137' '158' '167' '94' '107' '230' '49' '75'  
   '91' '122' '67' '83' '78' '52' '61' '93' '148' '129' '96' '71' '98' '115'  
   '53' '81' '79' '120' '152' '102' '108' '68' '58' '149' '89' '63' '48'  
   '66' '139' '103' '125' '133' '138' '135' '142' '77' '62' '132' '84' '64'  
   '74' '116' '82']
```

```
✓ [40] print(df.dtypes)
```

```
↔ mpg           float64  
   cylinders     int64  
   displacement  float64  
   horsepower    object  
   weight        int64  
   acceleration  float64  
   model year    int64  
   origin        int64  
   car name      object  
   dtype: object
```


데이터 표준화

- 자료형 변환

- 숫자가 문자열object로 저장된 경우가 있음
 - horsepower 의 '?' 값을 NaN으로 치환하고 해당 열을 삭제하기
 - horsepower의 속성을 float로 바꾸기

```
✓ [42] import numpy as np
0초

df.replace({'horsepower': '?'}, np.nan, inplace = True)
df.dropna(subset = ['horsepower'], axis = 0, inplace = True)
df['horsepower'] = df['horsepower'].astype('float')

print(df['horsepower'].dtypes)
```

↔ float64

데이터 표준화

- 자료형 변환

- origin 정수형 데이터 (1, 2, 3)를 실제 국가 이름인 USA, EU, JPN으로 변경하기
 - origin에 str 값을 넣으면 type이 object로 바뀜

```
✓ [43] print(df['origin'].unique())  
0초  
  
df['origin'].replace({1: 'USA', 2: 'EU', 3: 'JPN'}, inplace = True)  
  
print(df['origin'].unique())  
print(df['origin'].dtypes)
```

↕

```
[1 3 2]  
['USA' 'JPN' 'EU']  
object
```

데이터 표준화

- 자료형 변환

- origin 의 데이터 타입 변경하기

- category → object

유한한 범주형

✓
0초

```
[10] df['origin'] = df['origin'].astype('category')  
print(df['origin'].dtypes)
```

```
df['origin'] = df['origin'].astype('str')  
print(df['origin'].dtypes)
```



category
object

데이터 표준화

- 자료형 변환

- model year의 데이터 타입 변경하기

- 연도는 시간적 순서의 의미는 있으나 상대적인 크기는 별의미가 없음
 - Int64 → category // 년도는 범주형으로 표현하는 것이 적절함

✓
0초

```
[11] print(df['model year'].sample(3))  
df['model year'] = df['model year'].astype('category')  
print(df['model year'].sample(3))
```



47	71
387	82
337	80

Name: model year, dtype: int64

117	73
13	70
5	70

Name: model year, dtype: category

Categories (13, int64): [70, 71, 72, 73, ..., 79, 80, 81, 82]

범주형 (카테고리) 데이터 처리

- 구간 분할

- 구간 분할binning

- 연속 변수를 일정한 구간으로 나누고, 각 구간을 범주형 이산 변수로 변환하는 과정
 - 판다스 cut() 함수를 이용

✓
0초

```
[3] import pandas as pd
import numpy as np

df = pd.read_csv('auto-mpg.csv')

df.columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
              'acceleration', 'model year', 'origin', 'name']

df['horsepower'].replace('?', np.nan, inplace = True)
df.dropna(subset = ['horsepower'], axis = 0, inplace = True)
df['horsepower'] = df['horsepower'].astype('float')

count, bin_dividers = np.histogram(df['horsepower'], bins = 3)
print(bin_dividers)
```

경계값 리스트를 반환함 (예, 46 ~ 107.3 구간)



```
[ 46.          107.33333333 168.66666667 230.          ]
```

✓
0초

```
[4] print(count)
```



```
[257 103  32]
```

범주형 (카테고리) 데이터 처리

- 구간 분할

- 구간 분할binning

- 판다스 cut() 함수를 이용

✓
0초

```
[7] bin_names = ['저출력', '보통출력', '고출력']

df['hp_bin'] = pd.cut(x = df['horsepower'],
                      bins = bin_dividers,
                      labels = bin_names,
                      include_lowest = True)

print(df[['horsepower', 'hp_bin']].head(15))
```

경계값 포함 여부

```
horsepower hp_bin
0         130.0  보통출력
1         165.0  보통출력
2         150.0  보통출력
3         150.0  보통출력
4         140.0  보통출력
5         198.0   고출력
6         220.0   고출력
7         215.0   고출력
8         225.0   고출력
9         190.0   고출력
10        170.0   고출력
11         160.0  보통출력
12         150.0  보통출력
13         225.0   고출력
14          95.0  저출력
```

범주형 (카테고리) 데이터 처리

- 더미 변수

- 더미 변수 dummy variable

- 범주형 데이터는 머신러닝 알고리즘에 바로 쓸 수가 없는데, 인식 가능한 숫자 0 또는 1로 표현함
 - hp_bin 이 해당되면 1, 아니면 0 ()

범주형 (카테고리) 데이터 처리

- 더미 변수

- 더미 변수 dummy variable

- 범주형 데이터는 머신러닝 알고리즘에 바로 쓸 수가 없는데, 인식 가능한 숫자 0 또는 1로 표현함
 - hp_bin 이 해당되면 1, 아니면 0 (원핫인코딩 one-hot-encoding)

✓
0초

```
[9] horsepower_dummies = pd.get_dummies(df['hp_bin'])  
    print(horsepower_dummies.head(15))
```



	저 출력	보통 출력	고 출력
0	False	True	False
1	False	True	False
2	False	True	False
3	False	True	False
4	False	True	False
5	False	False	True
6	False	False	True
7	False	False	True
8	False	False	True
9	False	False	True
10	False	False	True
11	False	True	False
12	False	True	False
13	False	False	True
14	True	False	False

범주형 (카테고리) 데이터 처리

- 더미 변수

- 더미 변수 dummy variable

- 왜 원핫인코딩이 안되었을까요?
 - horsepower가 float 타입이어서

✓
0초

```
[25] bin_names = ['저출력', '보통출력', '고출력']

df['hp_bin'] = pd.cut(x = df['horsepower'],
                      bins = bin_dividers,
                      labels = bin_names,
                      include_lowest = True)

horsepower_dummies = pd.get_dummies(df['hp_bin'], dtype=int)
print(horsepower_dummies.head(15))
```



	저출력	보통출력	고출력
0	0	1	0
1	0	1	0
2	0	1	0
3	0	1	0
4	0	1	0
5	0	0	1
6	0	0	1
7	0	0	1
8	0	0	1
9	0	0	1
10	0	0	1
11	0	1	0
12	0	1	0
13	0	0	1
14	1	0	0

범주형 (카테고리) 데이터 처리

- 원핫인코딩

- sklearn 라이브러리를 이용하면 됨

- from sklearn import preprocessing

```
✓ [26] from sklearn import preprocessing
0초 label_encoder = preprocessing.LabelEncoder()
onehot_encoder = preprocessing.OneHotEncoder()

onehot_labeled = label_encoder.fit_transform(df['hp_bin'].head(15))
print(onehot_labeled)

onehot_reshaped = onehot_labeled.reshape(len(onehot_labeled), 1)
print(onehot_reshaped)
print(type(onehot_reshaped))

onehot_fitted = onehot_encoder.fit_transform(onehot_reshaped)
print(onehot_fitted)
print(type(onehot_fitted))
```

2차원 행렬로 변경

희소행렬로 변경
- 행·열 좌표 및 값

```
→ [1 1 1 1 1 0 0 0 0 0 0 1 1 0 2]
[[1]
 [1]
 [1]
 [1]
 [1]
 [0]
 [0]
 [0]
 [0]
 [0]
 [0]
 [1]
 [1]
 [0]
 [2]]
<class 'numpy.ndarray'>
<Compressed Sparse Row sparse matrix of dtype 'float64'
  with 15 stored elements and shape (15, 3)>

Coords      Values
(0, 1)       1.0
(1, 1)       1.0
(2, 1)       1.0
(3, 1)       1.0
(4, 1)       1.0
(5, 0)       1.0
(6, 0)       1.0
(7, 0)       1.0
(8, 0)       1.0
(9, 0)       1.0
(10, 0)      1.0
(11, 1)      1.0
(12, 1)      1.0
(13, 0)      1.0
(14, 2)      1.0
<class 'scipy.sparse._csr.csr_matrix'>
```

정규화

- 데이터의 상대적인 크기

- 정규화normalization

- 데이터 값을 동일한 크기 기준으로 나눈 비율로 나타내는 것
- 일반적으로 자기자신을 최대값으로 나누면 1보다 작음

```
✓ [27] df['horsepower'].replace('?', np.nan, inplace = True)
0초 df.dropna(subset = ['horsepower'], axis = 0, inplace = True)
df['horsepower'] = df['horsepower'].astype('float')

print(df.horsepower.describe())
print('##n')

df.horsepower = df.horsepower / abs(df.horsepower.max())

print(df.horsepower.head()) 정규화 > 0 이어야 함
print('##n')
print(df.horsepower.describe())
```

정규화

- 데이터의 상대적인 크기

- 정규화normalization

- 데이터 값을 동일한 크기 기준으로 나눈 비율로 나타내는 것
- 일반적으로 자기자신을 최대값으로 나누면 1보다 작음

```
↔ count    392.000000
   mean     104.469388
   std       38.491160
   min       46.000000
  25%       75.000000
  50%       93.500000
  75%      126.000000
  max      230.000000
Name: horsepower, dtype: float64
```

```
0    0.565217
1    0.717391
2    0.652174
3    0.652174
4    0.608696
Name: horsepower, dtype: float64
```

```
count    392.000000
mean       0.454215
std        0.167353
min        0.200000
25%        0.326087
50%        0.406522
75%        0.547826
max        1.000000
Name: horsepower, dtype: float64
```

정규화

- 데이터의 상대적인 크기

- 정규화normalization

- 또 다른 방법으로, 각 열 데이터에서 열의 최소값을 뺀 것을 분자로 하고, 데이터 중 최대값과 최소값을 뺀 값으로 나누는 방법도 있음

✓
0초

```
[29] print(df.horsepower.describe())  
      print('\n')  
  
      min_x = df.horsepower - df.horsepower.min()  
      min_max = df.horsepower.max() - df.horsepower.min()  
      df.horsepower = min_x / min_max  
  
      print(df.horsepower.head())  
      print('\n')  
      print(df.horsepower.describe())
```

정규화

- 데이터의 상대적인 크기

- 정규화normalization

- 또 다른 방법으로, 각 열 데이터에서 열의 최소값을 뺀 것을 분자로 하고, 데이터 중 최대값과 최소값을 뺀

값으로 나누는 방법도 있음

```
count 392.000000
mean   104.469388
std     38.491160
min     46.000000
25%    75.000000
50%    93.500000
75%   126.000000
max   230.000000
Name: horsepower, dtype: float64
```

```
0    0.565217
1    0.717391
2    0.652174
3    0.652174
4    0.608696
Name: horsepower, dtype: float64
```

```
count 392.000000
mean   0.454215
std     0.167353
min     0.200000
25%    0.326087
50%    0.406522
75%    0.547826
max     1.000000
Name: horsepower, dtype: float64
```



```
count 392.000000
mean    0.454215
std     0.167353
min     0.200000
25%    0.326087
50%    0.406522
75%    0.547826
max     1.000000
Name: horsepower, dtype: float64
```

```
0    0.456522
1    0.646739
2    0.565217
3    0.565217
4    0.510870
Name: horsepower, dtype: float64
```

```
count 392.000000
mean   0.317768
std     0.209191
min     0.000000
25%    0.157609
50%    0.258152
75%    0.434783
max     1.000000
Name: horsepower, dtype: float64
```

정규화 방법에 따라 정규화
데이터 간의 상관도에 영향
을 줌

시계열 데이터

※ 출처: <https://javapp.tistory.com/167>

- 다른 자료형을 시계열 객체로 변환

- 문자열을 Timestamp로 변환

- 판다스 to_datetime() 함수를 사용하면 판다스 timestamp로 나타낼 수 있음 0초

- stock-data.csv를 colab에 업로드함

```
[30] df = pd.read_csv('stock-data.csv')

print(df.head())
print('\n')
print(df.info())
```



	Date	Close	Start	High	Low	Volume
0	2018-07-02	10100	10850	10900	10000	137977
1	2018-06-29	10700	10550	10900	9990	170253
2	2018-06-28	10400	10900	10950	10150	155769
3	2018-06-27	10900	10800	11050	10500	133548
4	2018-06-26	10800	10900	11000	10700	63039

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Date    20 non-null      object
1   Close   20 non-null      int64
2   Start   20 non-null      int64
3   High    20 non-null      int64
4   Low     20 non-null      int64
5   Volume  20 non-null      int64
dtypes: int64(5), object(1)
memory usage: 1.1+ KB
None
```

시계열 데이터

- 다른 자료형을 시계열 객체로 변환
 - 문자열을 Timestamp로 변환
 - 판다스 to_datetime() 함수를 사용하면
 - Timestamp 객체로 바뀜

✓
0초

```
[31] df['new_Date'] = pd.to_datetime(df['Date'])
```

```
print(df.head())  
print(df.info())  
print(type(df['new_Date'][0]))
```



```
      Date  Close  Start  High   Low  Volume  new_Date  
0  2018-07-02  10100  10850  10900  10000  137977  2018-07-02  
1  2018-06-29  10700  10550  10900   9990  170253  2018-06-29  
2  2018-06-28  10400  10900  10950  10150  155769  2018-06-28  
3  2018-06-27  10900  10800  11050  10500  133548  2018-06-27  
4  2018-06-26  10800  10900  11000  10700   63039  2018-06-26
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 20 entries, 0 to 19
```

```
Data columns (total 7 columns):
```

#	Column	Non-Null Count	Dtype
0	Date	20 non-null	object
1	Close	20 non-null	int64
2	Start	20 non-null	int64
3	High	20 non-null	int64
4	Low	20 non-null	int64
5	Volume	20 non-null	int64
6	new_Date	20 non-null	datetime64[ns]

```
dtypes: datetime64[ns](1), int64(5), object(1)
```

```
memory usage: 1.2+ KB
```

```
None
```

```
<class 'pandas._libs.tslibs.timestamps.Timestamp'>
```


시계열 데이터

- 다른 자료형을 시계열 객체로 변환
 - 열 인덱스 변경
 - 시간 순서에 맞춰 인덱싱 또는 슬라이싱 하기 용이

```
✓ [32] df.set_index('new_Date', inplace = True)
0초 df.drop('Date', axis = 1, inplace = True)

print(df.head())
print('\n')
print(df.info())
```



	Close	Start	High	Low	Volume
new_Date					
2018-07-02	10100	10850	10900	10000	137977
2018-06-29	10700	10550	10900	9990	170253
2018-06-28	10400	10900	10950	10150	155769
2018-06-27	10900	10800	11050	10500	133548
2018-06-26	10800	10900	11000	10700	63039

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 20 entries, 2018-07-02 to 2018-06-01
Data columns (total 5 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Close   20 non-null       int64
1   Start   20 non-null       int64
2   High    20 non-null       int64
3   Low     20 non-null       int64
4   Volume  20 non-null       int64
dtypes: int64(5)
memory usage: 960.0 bytes
None
```

시계열 데이터

※ 출처: <https://javapp.tistory.com/167>

- 다른 자료형을 시계열 객체로 변환
 - Timestamp를 Period (두 시점 사이의 기간)로 변환
 - 시간 순서에 맞춰 인덱싱 또는 슬라이싱 하기 용이

✓
0초 [33] dates = ['2019-01-01', '2020-03-01', '2021-06-01']

```
ts_dates = pd.to_datetime(dates)
print(ts_dates)
print('\n\n')

pr_day = ts_dates.to_period(freq = 'D')
print(pr_day)
pr_month = ts_dates.to_period(freq = 'M')
print(pr_month)
pr_year = ts_dates.to_period(freq = 'A')
print(pr_year)
```

⇒ DatetimeIndex(['2019-01-01', '2020-03-01', '2021-06-01'], dtype='datetime64[ns]', freq=None)

```
PeriodIndex(['2019-01-01', '2020-03-01', '2021-06-01'], dtype='period[D]')
PeriodIndex(['2019-01', '2020-03', '2021-06'], dtype='period[M]')
PeriodIndex(['2019', '2020', '2021'], dtype='period[Y-DEC]')
```

시계열 데이터

※ 출처: https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html

- 다른 자료형을 시계열 객체로 변환
 - Frequency 옵션의 종류

Date Offset	Frequency String	Description
<code>DateOffset</code>	None	Generic offset class, defaults to absolute 24 hours
<code>BDay</code> or <code>BusinessDay</code>	'B'	business day (weekday)
<code>CDay</code> or <code>CustomBusinessDay</code>	'C'	custom business day
<code>Week</code>	'W'	one week, optionally anchored on a day of the week
<code>WeekOfMonth</code>	'WOM'	the x-th day of the y-th week of each month
<code>LastWeekOfMonth</code>	'LWOM'	the x-th day of the last week of each month
<code>MonthEnd</code>	'ME'	calendar month end
<code>MonthBegin</code>	'MS'	calendar month begin
<code>BMonthEnd</code> or <code>BusinessMonthEnd</code>	'BME'	business month end
<code>BMonthBegin</code> or <code>BusinessMonthBegin</code>	'BMS'	business month begin
<code>CBMonthEnd</code> or <code>CustomBusinessMonthEnd</code>	'CBME'	custom business month end

시계열 데이터

※ 출처: https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html

- 시계열 데이터 만들기

- Timestamp 배열

- data_range() 함수를 사용하며 여러 개의 날짜Timestamp 가 들어 있는 배열 형태의 시계열 데이터를 만듦

```
✓ [35] ts_ms = pd.date_range(start = '2019-01-01', end = None, periods = 6, freq = 'MS', tz = 'Asia/Seoul')  
0초 print(ts_ms)
```

날짜 범위 끝을 정하지 않음

월의 시작일

```
↔ DatetimeIndex(['2019-01-01 00:00:00+09:00', '2019-02-01 00:00:00+09:00',  
                 '2019-03-01 00:00:00+09:00', '2019-04-01 00:00:00+09:00',  
                 '2019-05-01 00:00:00+09:00', '2019-06-01 00:00:00+09:00'],  
                 dtype='datetime64[ns, Asia/Seoul]', freq='MS')
```

시계열 데이터

※ 출처: https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html

• 시계열 데이터 만들기

▪ Timestamp 배열

- `date_range()` 함수를 사용하며 여러 개의 날짜Timestamp 가 들어 있는 배열 형태의 시계열 데이터를 만듦
- 'M': 월의 마지막 날짜
- '3M': 3개월 간격의 마지막 날짜

```
✓ [36] ts_m = pd.date_range(start = '2019-01-01', periods = 6, freq = 'M', tz = 'Asia/Seoul')  
0초 print(ts_m)  
  
ts_3m = pd.date_range(start = '2019-01-01', periods = 6, freq = '3M', tz = 'Asia/Seoul')  
print(ts_3m)
```

```
⇒ DatetimeIndex(['2019-01-31 00:00:00+09:00', '2019-02-28 00:00:00+09:00',  
                 '2019-03-31 00:00:00+09:00', '2019-04-30 00:00:00+09:00',  
                 '2019-05-31 00:00:00+09:00', '2019-06-30 00:00:00+09:00'],  
                dtype='datetime64[ns, Asia/Seoul]', freq='ME')  
DatetimeIndex(['2019-01-31 00:00:00+09:00', '2019-04-30 00:00:00+09:00',  
                 '2019-07-31 00:00:00+09:00', '2019-10-31 00:00:00+09:00',  
                 '2020-01-31 00:00:00+09:00', '2020-04-30 00:00:00+09:00'],  
                dtype='datetime64[ns, Asia/Seoul]', freq='3ME')
```

시계열 데이터

※ 출처: https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html

- 시계열 데이터 만들기

- Period

- period_range() 함수를 사용하며 여러 개의 기간^{Period} 이 들어 있는 시계열 데이터를 만들

✓
0초

```
[37] pr_m = pd.period_range(start = '2019-01-01', end = None, periods = 6, freq = 'M')  
      print(pr_m)
```



```
PeriodIndex(['2019-01', '2019-02', '2019-03', '2019-04', '2019-05', '2019-06'], dtype='period[M]')
```

✓
0초

```
[38] pr_h = pd.period_range(start = '2019-01-01', end = None, periods = 3, freq = 'H')  
      print(pr_h)  
      print('#n')  
      pr_2h = pd.period_range(start = '2019-01-01', end = None, periods = 3, freq = '2H')  
      print(pr_2h)
```



```
PeriodIndex(['2019-01-01 00:00', '2019-01-01 01:00', '2019-01-01 02:00'], dtype='period[h]')
```

```
PeriodIndex(['2019-01-01 00:00', '2019-01-01 02:00', '2019-01-01 04:00'], dtype='period[2h]')
```

시계열 데이터

- 시계열 데이터 활용

- 날짜 데이터 분리

- 연-월-일 날짜 데이터에서 일부를 분리할 수 있음

```
✓ [41] df = pd.read_csv('stock-data.csv')  
0초  
  
df['new_Date'] = pd.to_datetime(df['Date'])  
print(df.head())  
print('#n')  
  
df['Year'] = df['new_Date'].dt.year  
df['Month'] = df['new_Date'].dt.month  
df['Day'] = df['new_Date'].dt.day  
print(df.head())
```



	Date	Close	Start	High	Low	Volume	new_Date
0	2018-07-02	10100	10850	10900	10000	137977	2018-07-02
1	2018-06-29	10700	10550	10900	9990	170253	2018-06-29
2	2018-06-28	10400	10900	10950	10150	155769	2018-06-28
3	2018-06-27	10900	10800	11050	10500	133548	2018-06-27
4	2018-06-26	10800	10900	11000	10700	63039	2018-06-26

	Date	Close	Start	High	Low	Volume	new_Date	Year	Month	Day
0	2018-07-02	10100	10850	10900	10000	137977	2018-07-02	2018	7	2
1	2018-06-29	10700	10550	10900	9990	170253	2018-06-29	2018	6	29
2	2018-06-28	10400	10900	10950	10150	155769	2018-06-28	2018	6	28
3	2018-06-27	10900	10800	11050	10500	133548	2018-06-27	2018	6	27
4	2018-06-26	10800	10900	11000	10700	63039	2018-06-26	2018	6	26

시계열 데이터

- 시계열 데이터 활용

- 날짜 데이터 분리

- 'A': 연도
- 'M': 연-월

✓
0초

```
[42] df['Date_yr'] = df['new_Date'].dt.to_period(freq = 'A')  
df['Date_m'] = df['new_Date'].dt.to_period(freq = 'M')  
print(df.head())
```



	Date	Close	Start	High	Low	Volume	new_Date	Year	Month	#
0	2018-07-02	10100	10850	10900	10000	137977	2018-07-02	2018	7	
1	2018-06-29	10700	10550	10900	9990	170253	2018-06-29	2018	6	
2	2018-06-28	10400	10900	10950	10150	155769	2018-06-28	2018	6	
3	2018-06-27	10900	10800	11050	10500	133548	2018-06-27	2018	6	
4	2018-06-26	10800	10900	11000	10700	63039	2018-06-26	2018	6	

	Day	Date_yr	Date_m
0	2	2018	2018-07
1	29	2018	2018-06
2	28	2018	2018-06
3	27	2018	2018-06
4	26	2018	2018-06

시계열 데이터

- 시계열 데이터 활용

- 날짜 데이터 분리

- Date_m을 새로운 열 인덱스로 지정함

✓
0초

```
[43] df.set_index('Date_m', inplace = True)  
      print(df.head())
```



	Date	Close	Start	High	Low	Volume	new_Date	Year	#
Date_m									
2018-07	2018-07-02	10100	10850	10900	10000	137977	2018-07-02	2018	
2018-06	2018-06-29	10700	10550	10900	9990	170253	2018-06-29	2018	
2018-06	2018-06-28	10400	10900	10950	10150	155769	2018-06-28	2018	
2018-06	2018-06-27	10900	10800	11050	10500	133548	2018-06-27	2018	
2018-06	2018-06-26	10800	10900	11000	10700	63039	2018-06-26	2018	

	Month	Day	Date_yr
Date_m			
2018-07	7	2	2018
2018-06	6	29	2018
2018-06	6	28	2018
2018-06	6	27	2018
2018-06	6	26	2018

시계열 데이터

- 시계열 데이터 활용

- 날짜 인덱스 활용

- new_Date를 새로운 열 인덱스로 지정함

✓
0초

```
[47] df = pd.read_csv('stock-data.csv')

df['new_Date'] = pd.to_datetime(df['Date'])
df.set_index('new_Date', inplace = True)

print(df.head())
print('\n')
print(df.index)
```



	Date	Close	Start	High	Low	Volume
new_Date						
2018-07-02	2018-07-02	10100	10850	10900	10000	137977
2018-06-29	2018-06-29	10700	10550	10900	9990	170253
2018-06-28	2018-06-28	10400	10900	10950	10150	155769
2018-06-27	2018-06-27	10900	10800	11050	10500	133548
2018-06-26	2018-06-26	10800	10900	11000	10700	63039

```
DatetimeIndex(['2018-07-02', '2018-06-29', '2018-06-28', '2018-06-27',
                '2018-06-26', '2018-06-25', '2018-06-22', '2018-06-21',
                '2018-06-20', '2018-06-19', '2018-06-18', '2018-06-15',
                '2018-06-14', '2018-06-12', '2018-06-11', '2018-06-08',
                '2018-06-07', '2018-06-05', '2018-06-04', '2018-06-01'],
              dtype='datetime64[ns]', name='new_Date', freq=None)
```

시계열 데이터

- 시계열 데이터 활용

- 날짜 인덱스 활용

- new_Date라는 날짜 인덱스를 이용하여 원하는 데이터 출력

```
✓ [54] df_y = df.loc['2018']  
0초 print(df_y.head())  
print('\n')  
df_ym = df.loc['2018-07']  
print(df_ym)  
print('\n')  
df_ym_cols = df.loc['2018-07', 'Start':'High']  
print(df_ym_cols)  
print('\n')  
df_ymd = df.loc['2018-07-02']  
print(df_ymd)  
print('\n')  
df_ymd_range = df.loc[(df.index >= '2018-06-20') & (df.index <= '2018-06-25')]  
print(df_ymd_range)
```

시계열 데이터

- 시계열 데이터 활용

- 날짜 인덱스 활용

- new_Date라는 날짜 인덱스를 이용하여 원하는 데이터 출력



	Date	Close	Start	High	Low	Volume
new_Date						
2018-07-02	2018-07-02	10100	10850	10900	10000	137977
2018-06-29	2018-06-29	10700	10550	10900	9990	170253
2018-06-28	2018-06-28	10400	10900	10950	10150	155769
2018-06-27	2018-06-27	10900	10800	11050	10500	133548
2018-06-26	2018-06-26	10800	10900	11000	10700	63039

	Date	Close	Start	High	Low	Volume
new_Date						
2018-07-02	2018-07-02	10100	10850	10900	10000	137977

	Start	High
new_Date		
2018-07-02	10850	10900

```
Date      2018-07-02
Close      10100
Start      10850
High       10900
Low        10000
Volume     137977
Name: 2018-07-02 00:00:00, dtype: object
```

	Date	Close	Start	High	Low	Volume
new_Date						
2018-06-25	2018-06-25	11150	11400	11450	11000	55519
2018-06-22	2018-06-22	11300	11250	11450	10750	134805
2018-06-21	2018-06-21	11200	11350	11750	11200	133002
2018-06-20	2018-06-20	11550	11200	11600	10900	308596

시계열 데이터

- 시계열 데이터 활용

- 날짜 인덱스 활용

- new_Date라는 날짜 인덱스를 이용하여 원하는 데이터 출력

✓
0초

```
[56] today = pd.to_datetime('2018-12-25')  
      df['time_delta'] = today - df.index  
      df.set_index('time_delta', inplace = True)  
      df_180 = df['180 days':'189 days']  
      print(df_180)
```



	Date	Close	Start	High	Low	Volume
time_delta						
180 days	2018-06-28	10400	10900	10950	10150	155769
181 days	2018-06-27	10900	10800	11050	10500	133548
182 days	2018-06-26	10800	10900	11000	10700	63039
183 days	2018-06-25	11150	11400	11450	11000	55519
186 days	2018-06-22	11300	11250	11450	10750	134805
187 days	2018-06-21	11200	11350	11750	11200	133002
188 days	2018-06-20	11550	11200	11600	10900	308596
189 days	2018-06-19	11300	11850	11950	11300	180656

Wrap-Up

- 배운 것들을 정리해봅시다.

감사합니다.

